

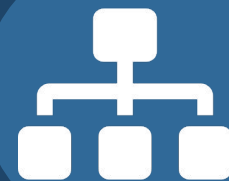
CURSO DE EXTENSÃO · PROGRAMAÇÃO PARA INICIANTES

Aula 02

Lógica de Programação

De um problema mal definido até uma sequência de passos organizada — e como representar esse raciocínio visualmente antes de escrever a primeira linha em Python.

Pré-requisito: Aula 01 — Introdução à Programação



O que vamos ver hoje

Do problema à representação visual: o caminho completo antes de programar de verdade.



Da necessidade ao problema

Transformar um pedido vago em algo que dá para resolver.



Pensar logicamente

Uma estratégia de 5 passos para organizar qualquer solução.



Decomposição

Dividir o problema grande em tarefas pequenas e simples.



Algoritmo (uma prévia)

O que é, e por que trabalharemos direto com Python.



Fluxogramas

Representar visualmente um processo antes do código.



Exemplo prático

Média de 3 notas para 3 alunos, do fluxograma à tabela.

Da necessidade ao problema

Programar começa antes de abrir uma IDE: começa por entender qual problema precisa ser resolvido.



Descrição vaga:

“Calcular as médias da turma.”

- Quantos alunos serão avaliados?
- Quantas notas cada aluno possui?
- Como a média será calculada?
- O que deve ser apresentado ao final?

PROBLEMA DESTA AULA

Uma turma tem 3 alunos. Cada aluno possui 3 notas.
Precisamos calcular e mostrar a média de cada um.



Por que esclarecer antes de programar?

Um problema mal definido gera um programa que resolve a coisa errada — mesmo que o código esteja "correto".

Clarear o pedido é o primeiro passo da lógica de programação — antes de qualquer linha de código.

Pensar logicamente para resolver

Organizar ações de maneira coerente: o que precisa acontecer primeiro, que informações são necessárias, como chegar ao resultado.



Essa sequência funciona para qualquer problema — da média de notas ao sistema mais complexo.

Decomposição: dividir para conquistar

Um problema grande pode ser dividido em tarefas menores. Cada etapa é simples; juntas, resolvem o problema completo.



Problema grande

Calcular e mostrar a média de 3 alunos

- 1 Obter as três notas do aluno
- 2 Somar as notas
- 3 Dividir a soma por 3
- 4 Apresentar a média
- 5 Repetir para os demais alunos



Por que dividir ajuda?

- Cada tarefa pequena é mais fácil de entender e testar
- Erros ficam mais fáceis de localizar
- Tarefas repetidas (como “calcular 1 média”) só precisam ser pensadas uma vez
- É a base para, mais adiante, usar repetições no código

Algoritmo: uma prévia

Um algoritmo é uma **sequência finita de passos, organizados logicamente, com a finalidade de chegar a uma solução.**

Exemplo: média de 3 notas

1 Receber três notas

2 Somar as notas

3 Dividir a soma por três

4 Mostrar a média

Todo algoritmo precisa ter:



Passos compreensíveis



Ordem coerente entre os passos



Um ponto de término

Veremos algoritmos com mais profundidade na próxima aula.

Por que não usar Portugol e VisualG?

Em muitos cursos introdutórios, a lógica é ensinada com “pseudo-código” em Portugol uma representação didática de algoritmos, geralmente praticada na ferramenta VisualG.



O problema de usar Portugol primeiro

Como a carga horária é curta, aprender uma linguagem didática e depois migrar para outra exigiria tempo adicional de adaptação.

Portugol e VisualG não são inúteis — são recursos didáticos válidos; apenas não serão a ferramenta escolhida neste curso.



Por que ir direto ao Python

- Estudamos os fundamentos praticando na linguagem real do curso
- Python é usado em automação, análise de dados e desenvolvimento
- Um só caminho de aprendizado, sem etapa de tradução extra

“A lógica não pertence a uma linguagem específica. O mesmo raciocínio pode ser representado em fluxograma, Portugol, Python, C ou outras linguagens.”

O computador continua sendo literal

Como vimos na aula anterior, o computador não interpreta nossa intenção: executa instruções conforme foram definidas.

Suficiente para uma pessoa:

“Calcule a média.”



Um programa precisa de instruções explícitas:

Quais valores? Qual operação? Quantas vezes? Como mostrar?

Instruções explícitas que a lógica de programação organiza:



Quais valores utilizar



Qual operação realizar



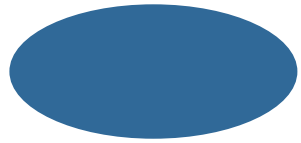
Quantas vezes executar



Como apresentar o resultado

Fluxogramas: os símbolos principais

Um fluxograma representa visualmente as etapas de um processo, por meio de símbolos conectados por setas.



Oval / terminador

Início ou fim

Marca os limites do processo



Paralelogramo

Entrada ou saída

Ler dados ou apresentar resultados



Retângulo

Processamento

Calcular ou atribuir valores



Losango

Decisão

Avaliar uma condição e escolher um caminho

A seta indica o fluxo: a sequência das etapas do início ao fim.

O fluxo básico de um processo

Nem todo algoritmo precisa de decisão — mas quase todos seguem esta espinha dorsal.

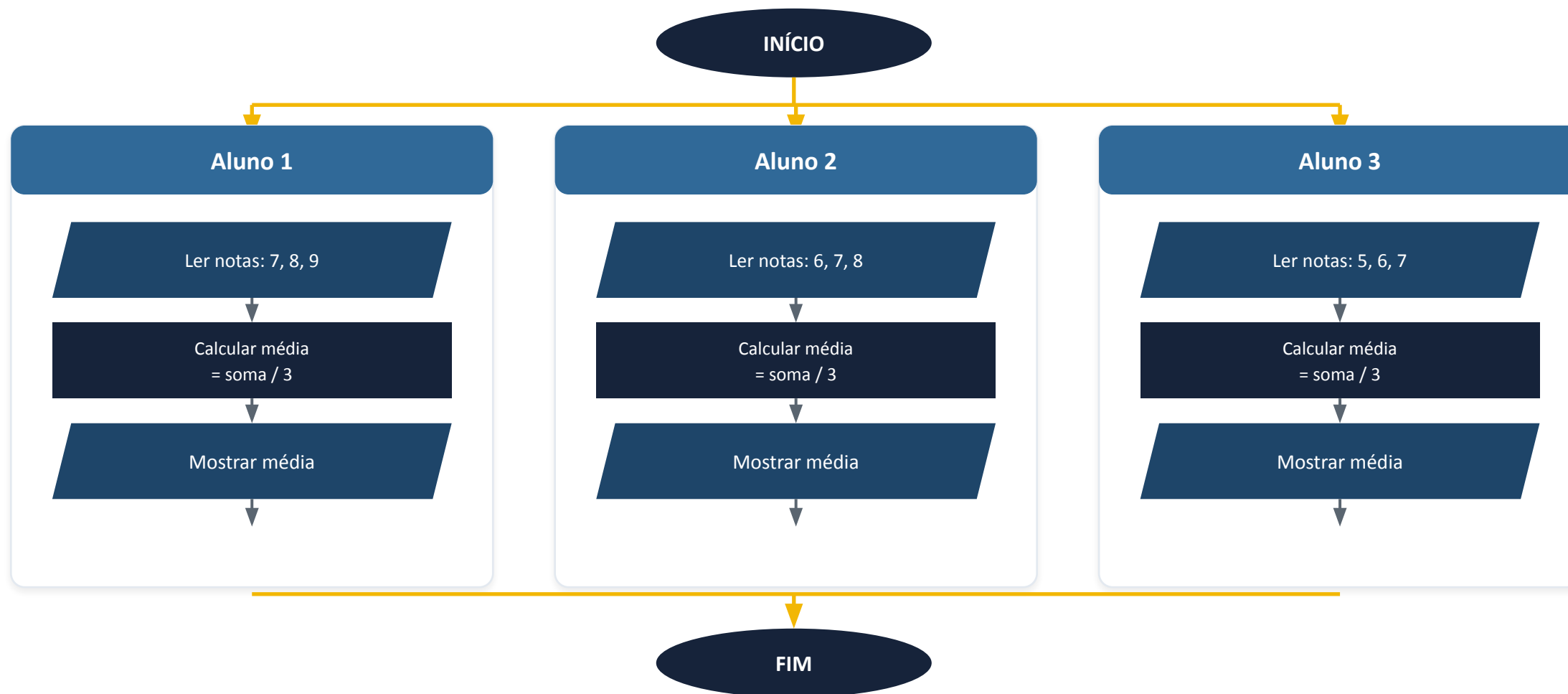


Exemplo aplicado

- Início do programa
- Ler as 3 notas
- Calcular a média
- Mostrar a média
- Fim do programa

Média de 3 notas para 3 alunos

O mesmo procedimento se repete três vezes — uma vez para cada aluno. Ainda sem estruturas de repetição formais.



Exemplo numérico

Regras: 3 alunos, 3 notas cada. Média = soma das notas ÷ 3.

Aluno	Nota 1	Nota 2	Nota 3	Média
1	7	8	9	8,0
2	6	7	8	7,0
3	5	6	7	6,0

O procedimento se repete três vezes: mesma lógica, dados diferentes.

Mais adiante, veremos como estruturas de repetição evitam repetir esse código manualmente.

O que levamos desta aula



Compreender antes de programar

Resolver um problema começa por entendê-lo de verdade.



Dados e resultado esperado

Precisam ser identificados antes de qualquer solução.



Decomposição

Dividir um problema em tarefas menores facilita sua solução.



Algoritmo

Organiza passos finitos numa sequência lógica.



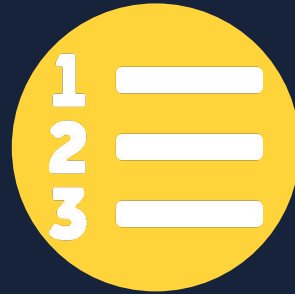
Fluxogramas

Representam visualmente um processo antes do código.



Python direto

A lógica vale para qualquer linguagem — usaremos Python, sem etapa em Portugol.



Próxima aula: Algoritmos

Vamos aprofundar como descrever soluções passo a passo e transformar o raciocínio em uma representação que possa ser implementada em Python.

Obrigado pela atenção e até a próxima aula!